TRƯỜNG ĐẠI HỌC FPT

Bộ môn Công nghệ Phần mềm

BÁO CÁO LAB 3

TÍCH HỢP FIREBASE & DỊCH VỤ ĐÁM MÂY

Ứng dụng Friggy - Quản lý Tủ lạnh Thông minh

Môn học: PRM393 - Mobile Programming

Sinh viên: Trần Quốc Lâm

Ứng dụng: Friggy (com.tqlam.friggy)

Firebase: friggy-61bee

Ngày nộp: 05/11/2026

Học kỳ Fall 2026

© 2026 Trần Quốc Lâm - FPT University

MỤC LỤC

1. Tổng quan dự án 3
2. Kiến trúc hệ thống 3
3. Firebase Authentication 4
4. Cloudinary - Cloud Storage 5
5. Firebase Cloud Messaging 6
6. Firebase Analytics 7
7. Firebase Crashlytics 8
8. Firebase Remote Config 9
9. Kế hoạch Analytics 10
10. Kết quả kiểm thử 11
11. Kết luận 12

© 2026 Trần Quốc Lâm - FPT University

1. TỔNG QUAN DỰ ÁN

Friggy là ứng dụng quản lý tủ lạnh thông minh, giúp người dùng theo dõi nguyên liệu, gợi ý công thức nấu ăn, lên kế hoạch bữa ăn hàng tuần và scan nguyên liệu bằng AI. Ứng dụng được phát triển bằng Flutter, kết nối với backend REST API tại api.friggy.io.vn.

Trong Lab 3, dự án tích hợp 6 dịch vụ Firebase và Cloud:

- Firebase Authentication - Đăng nhập Google, quản lý phiên
- Cloudinary (thay Cloud Storage) - Upload ảnh đại diện
- Firebase Cloud Messaging (FCM) - Thông báo đẩy
- Firebase Analytics - Đo lường hành vi (10 sự kiện)
- Firebase Crashlytics - Theo dõi và báo cáo lỗi
- Firebase Remote Config - Cấu hình từ xa (2 tham số)

2. KIẾN TRÚC HỆ THỐNG

Dự án tuân thủ kiến trúc MVVM + Provider. Giao diện (View/Screen) KHÔNG gọi trực tiếp Firebase SDK, mà chỉ tương tác thông qua các lớp Service:

View (Screen) -> Provider -> Service -> Firebase SDK

> Các Service đã tạo:

File Service	Chức năng
firebase_auth_service.dart	Bọc Firebase Auth - đăng nhập Google
cloudinary_service.dart	Bọc Cloudinary SDK - upload avatar
analytics_service.dart	Bọc Firebase Analytics - 10 sự kiện
crashlytics_service.dart	Bọc Crashlytics - ghi lỗi, custom keys
remote_config_service.dart	Bọc Remote Config - 2 tham số từ xa
notification_service.dart	Bọc FCM - push notification

Tất cả Service sử dụng Singleton pattern, khởi tạo trong main.dart trước khi runApp().

© 2026 Trần Quốc Lâm - FPT University

3. FIREBASE AUTHENTICATION

3.1 Mô tả

Firebase Authentication tích hợp đăng nhập Google Sign-In. Do ứng dụng đã có hệ thống xác thực backend riêng (REST API), Firebase Auth được triển khai dạng "dual auth" - đăng nhập đồng thời cả Firebase và backend.

3.2 Luồng hoạt động

- Bước 1: Người dùng bấm "Đăng nhập bằng Google"
- Bước 2: Google Sign-In SDK hiện dialog chọn tài khoản
- Bước 3: Nhận Google idToken -> gửi tới Firebase Auth tạo credential
- Bước 4: Đồng thời gửi idToken tới backend API để xác thực
- Bước 5: Lưu trạng thái đăng nhập và chuyển sang Trang chủ
- Bước 6: Khi đăng xuất: gọi Firebase signOut() + xóa token backend

3.3 Code tích hợp

firebase_auth_service.dart (trích)

```
class FirebaseAuthService {
  FirebaseAuthService._();
  static final instance = FirebaseAuthService._();
  final FirebaseAuth _auth = FirebaseAuth.instance;

  Future<UserCredential?> signInWithGoogle(GoogleSignInAccount googleUser) async {
    final googleAuth = await googleUser.authentication;
    final credential = GoogleAuthProvider.credential(
      idToken: googleAuth.idToken,
      accessToken: googleAuth.accessToken,
    );
    return await _auth.signInWithCredential(credential);
  }

  Stream<User?> get authStateChanges => _auth.authStateChanges();
  Future<void> signOut() => _auth.signOut();
}
```

3.4 Kiểm chứng

- Đăng nhập Google thành công -> Console > Authentication > Users hiện user
- Hủy đăng nhập (bấm back) -> app hiện thông báo, không crash
- Đăng xuất -> xóa trạng thái, quay về màn hình Login

© 2026 Trần Quốc Lâm - FPT University

4. CLOUDINARY - CLOUD STORAGE

4.1 Mô tả

Cloudinary được sử dụng thay thế Firebase Cloud Storage cho chức năng upload ảnh đại diện (avatar). Sử dụng unsigned upload preset để cho phép upload trực tiếp từ client.

4.2 Cấu hình

Tham số	Giá trị
Cloud Name	tqlam150504
Upload Preset	friggy_unsigned (Unsigned mode)
Folder	friggy/avatars/
Package	cloudinary_public ^0.23.1

4.3 Luồng Upload Avatar

- Người dùng vào Hồ sơ -> bấm icon camera trên avatar
- Chọn ảnh từ thư viện hoặc chụp mới (max 512x512, quality 85%)
- Upload song song: Cloudinary (lưu trữ cloud) + Backend API (cập nhật profile)
- Ảnh xuất hiện trong Cloudinary Dashboard -> friggy -> avatars

4.4 Code tích hợp

cloudinary_service.dart (trích)

```
class CloudinaryService {
  CloudinaryService._();
  static final instance = CloudinaryService._();
  CloudinaryPublic? _cloudinary;

  void initialize() {
    _cloudinary = CloudinaryPublic(
      AppConstants.cloudinaryCloudName,
      AppConstants.cloudinaryUploadPreset, cache: false);
  }

  Future<CloudinaryUploadResult?> uploadAvatar({
    required String filePath, required String userId,
  }) async {
    final response = await _cloudinary!.uploadFile(
      CloudinaryFile.fromFile(filePath,
        folder: 'friggy/avatars',
        resourceType: CloudinaryResourceType.Image));
    return CloudinaryUploadResult(
      secureUrl: response.secureUrl, publicId: response.publicId);
  }
}
```

© 2026 Trần Quốc Lâm - FPT University

5. FIREBASE CLOUD MESSAGING (FCM)

5.1 Mô tả

Firebase Cloud Messaging cho phép gửi thông báo đẩy (push notification) đến thiết bị người dùng. FCM đã được tích hợp sẵn trong dự án từ trước Lab 3, hỗ trợ cả foreground và background notification.

5.2 Tính năng

- Foreground: Hiển thị local notification khi app đang mở
- Background: Hiển thị notification trên system tray khi app chạy nền
- Data message: Xử lý payload tùy chỉnh từ server
- Token management: Tự động đăng ký FCM token với backend

5.3 Kiểm chứng

- Firebase Console -> Messaging -> Tạo campaign test -> Gửi notification
- App foreground: hiện local notification popup
- App background: hiện notification trên thanh trạng thái

6. FIREBASE ANALYTICS

6.1 Mô tả

Firebase Analytics được tích hợp để đo lường hành vi người dùng. Tổng cộng 10 sự kiện tùy chỉnh được ghi nhận, vượt yêu cầu tối thiểu 7 sự kiện của Lab 3.

6.2 Danh sách sự kiện

Sự kiện	Tham số	Mô tả
login	method	Đăng nhập (google/phone)
sign_up	method	Đăng ký tài khoản mới
logout	-	Đăng xuất
scan_ingredient	scan_type	Scan nguyên liệu (photo/receipt/barcode)
add_ingredient	name, category	Thêm nguyên liệu thủ công
view_recipe	recipe_name	Xem chi tiết công thức
generate_meal	ingredient_count	Tạo kế hoạch bữa ăn tuần
ai_chat_msg	session_id	Gửi tin nhắn cho AI Chef
upload_file	file_type, size_kb	Upload file (avatar)

© 2026 Trần Quốc Lâm - FPT University

view_statistics	stat_type	Xem thống kê nguyên liệu

© 2026 Trần Quốc Lâm - FPT University

7. FIREBASE CRASHLYTICS

7.1 Mô tả

Firebase Crashlytics theo dõi và báo cáo các lỗi ứng dụng (crash và non-fatal errors) lên Firebase Console. Service tự động bắt tất cả FlutterError và Platform errors, đồng thời gán user ID và custom keys.

7.2 Tính năng đã tích hợp

- Tự động ghi nhận tất cả Flutter errors (FlutterError.onError)
- Bắt lỗi ngoài Flutter zone (PlatformDispatcher.onError)
- Gán User ID: CrashlyticsService.setUser(userId, name, loginMethod)
- Custom Keys: user_name, login_method, app_version
- Non-fatal error: gửi lỗi test từ Settings screen
- Test crash: crash có chủ đích từ khu vực debug

7.3 Code tích hợp

crashlytics_service.dart (trích)

```
class CrashlyticsService {
  CrashlyticsService._();
  static final instance = CrashlyticsService._();
  FirebaseCrashlytics get _crashlytics => FirebaseCrashlytics.instance;

  Future<void> initialize() async {
    FlutterError.onError = (errorDetails) {
      _crashlytics.recordFlutterFatalError(errorDetails);
    };
    PlatformDispatcher.instance.onError = (error, stack) {
      _crashlytics.recordError(error, stack, fatal: true);
      return true;
    };
  }

  Future<void> setUserIdentifier(String uid) async {
    await _crashlytics.setUserIdentifier(uid);
  }
  Future<void> setCustomKey(String key, Object value) async {
    await _crashlytics.setCustomKey(key, value);
  }
  Future<void> recordError(dynamic e, StackTrace? s) async {
    await _crashlytics.recordError(e, s);
  }
  void triggerTestCrash() => _crashlytics.crash();
}
```

7.4 Kiểm chứng

- Firebase Console -> Crashlytics -> hiển thị 2 issues:

© 2026 Trần Quốc Lâm - FPT University

- Non-fatal: "CrashlyticsService.initialize" (FlutterError)
- Fatal crash: "FlutterFirebaseCrashlyticsPlugin.crash()" (test crash)
- Bấm vào issue -> thấy User ID, Custom Keys

© 2026 Trần Quốc Lâm - FPT University

8. FIREBASE REMOTE CONFIG

8.1 Mô tả

Firebase Remote Config cho phép thay đổi hành vi ứng dụng từ xa mà không cần phát hành bản cập nhật mới. Lab 3 yêu cầu tối thiểu 2 tham số tạo thay đổi có thể quan sát được.

8.2 Hai tham số đã cấu hình

Tham số	Kiểu	Mặc định	Tác dụng
ai_chat_enabled	bool	true	Bật/tắt tính năng AI Chat
max_scan_per_day	int	10	Giới hạn số lần scan/ngày

> ai_chat_enabled = false

Khi tắt: tab "Đầu bếp AI" trên bottom navigation hiện thông báo "AI Chat is under maintenance" thay vì màn hình chat.

> max_scan_per_day = 10

Giới hạn số lần scan nguyên liệu mỗi ngày. Giá trị hiển thị trong phần Firebase Debug của Settings screen.

8.3 Code tích hợp

remote_config_service.dart (trích)

```
class RemoteConfigService {
  RemoteConfigService._();
  static final instance = RemoteConfigService._();
  final _remoteConfig = FirebaseRemoteConfig.instance;

  Future<void> initialize() async {
    await _remoteConfig.setConfigSettings(RemoteConfigSettings(
      fetchTimeout: Duration(seconds: 10),
      minimumFetchInterval: Duration(seconds: 60)));
    await _remoteConfig.setDefaults({
      'ai_chat_enabled': true, 'max_scan_per_day': 10});
    await _remoteConfig.fetchAndActivate();
  }

  bool get isAiChatEnabled => _remoteConfig.getBool('ai_chat_enabled');
  int get maxScanPerDay => _remoteConfig.getInt('max_scan_per_day');
  Future<void> refresh() => _remoteConfig.fetchAndActivate();
}
```

© 2026 Trần Quốc Lâm - FPT University

9. KẾ HOẠCH ANALYTICS

Kế hoạch đo lường (Analytics Plan) được thiết kế để theo dõi các hành vi quan trọng của người dùng trong ứng dụng Friggy:

#	Sự kiện	Màn hình	Tham số	Mục đích
1	login	Login	method	Theo dõi phương thức đăng nhập
2	sign_up	Register	method	Đo tỷ lệ đăng ký mới
3	logout	Profile	-	Theo dõi đăng xuất
4	scan_ingredient	Scan *	scan_type	Đo tần suất sử dụng scan
5	add_ingredient	Manual Add	name, category	Theo dõi thêm nguyên liệu
6	view_recipe	Recipe Detail	recipe_name	Đo mức quan tâm công thức
7	generate_meal	Meal Plan	ingredient_count	Đo sử dụng meal plan
8	ai_chat_msg	AI Chat	session_id	Đo tương tác AI
9	upload_file	Profile	file_type, size	Theo dõi upload
10	view_stats	Statistics	stat_type	Đo xem thống kê

Tổng cộng 10 sự kiện tùy chỉnh, vượt yêu cầu tối thiểu 7 sự kiện của Lab 3. Mỗi sự kiện được ghi nhận thông qua AnalyticsService (không gọi trực tiếp SDK từ Screen), tuân thủ kiến trúc MVVM.

© 2026 Trần Quốc Lâm - FPT University

10. KẾT QUẢ KIỂM THỬ & BẰNG CHỨNG

Tóm tắt kết quả kiểm thử cho từng dịch vụ:

#	Dịch vụ	Kết quả	Chi tiết
1	Authentication	PASS	Google Sign-In OK, Console hiện user
2	Cloudinary	PASS	Avatar upload -> friggy/avatars/ trên Dashboard
3	Cloud Messaging	PASS	Push notification foreground + background
4	Analytics	PASS	DebugView hiện 10 sự kiện realtime
5	Crashlytics	PASS	Console hiện 2 issues: non-fatal + crash
6	Remote Config	PASS	Thay đổi Console -> app cập nhật ngay

10.1 Firebase Console Evidence

- Authentication: Users tab hiển thị tài khoản Google đã đăng nhập
- Crashlytics: Issues tab hiển thị 2 crash reports (fatal + non-fatal)
- Remote Config: 2 params published (ai_chat_enabled, max_scan_per_day)
- Analytics: DebugView hiện sự kiện realtime khi thao tác trên app
- Cloudinary: Dashboard -> friggy -> avatars -> ảnh avatar đã upload

10.2 Xử lý lỗi

Mỗi dịch vụ đều có cả kiểm thử lỗi theo yêu cầu Lab 3:

- Auth: Hủy đăng nhập Google -> hiện thông báo "Đăng nhập đã bị hủy"
- Cloudinary: Upload thất bại -> hiện SnackBar lỗi, app không crash
- FCM: Token không hợp lệ -> log lỗi, không ảnh hưởng app
- Analytics: Offline -> events cached, gửi khi có mạng
- Crashlytics: Non-fatal error -> ghi nhận lên Console, app vẫn chạy
- Remote Config: Fetch thất bại -> dùng giá trị mặc định

© 2026 Trần Quốc Lâm - FPT University

11. KẾT LUẬN

Lab 3 đã hoàn thành việc tích hợp 6 dịch vụ Firebase/Cloud vào ứng dụng Friggy, đáp ứng đầy đủ yêu cầu của môn PRM393:

- Kiến trúc MVVM + Provider: Tất cả Firebase SDK được bọc trong Service layer, Screen/View không gọi trực tiếp SDK.
- Firebase Authentication: Đăng nhập Google hoạt động ổn định, xử lý hủy đăng nhập và đăng xuất đúng cách.
- Cloudinary (Cloud Storage): Upload avatar thành công, ảnh lưu trong folder friggy/avatars trên Cloudinary Dashboard.
- Firebase Cloud Messaging: Push notification hoạt động cả foreground và background.
- Firebase Analytics: 10 sự kiện tùy chỉnh được ghi nhận, hiển thị trên DebugView.
- Firebase Crashlytics: Ghi nhận lỗi fatal và non-fatal, gán user ID và custom keys.
- Firebase Remote Config: 2 tham số tạo thay đổi quan sát được (bật/tắt AI Chat, giới hạn scan/ngày).
- Xử lý lỗi: Mỗi dịch vụ có cả kiểm thử lỗi, ứng dụng hiện thông báo rõ ràng.

--- Hết ---

© 2026 Trần Quốc Lâm - FPT University